

1. I have been assigned a project based on the documents that I will upload. My goal is to fully understand the requirements and then build an application that satisfies them.

Known constraints and expectations:

- The solution should be a Web Application with a Backend.
 - Preferred backend technology: Spring Boot.
 - Preferred frontend technology: Angular or React.
 - I will need to justify every major technical decision, including the architecture, frameworks, libraries, database choice, design patterns, deployment approach, and implementation details.
 - I am not experienced with this technology stack, so I need guidance suitable for a beginner while still maintaining industry-level standards.
2. How I want you to help:
 - First, read and analyze the docs thoroughly.
 - Explain the problem statement in simple, non-technical language.
 - Break down the requirements into functional requirements, non-functional requirements, assumptions, constraints, and open questions.
 - Before suggesting any architecture or implementation, ask me for confirmation that I understand the requirements.
 - Move step-by-step and do not skip ahead.
 - At the end of each step:
 - Summarize what we learned.
 - Explain why it matters.
 - Ask for my approval before proceeding to the next step.
 - When discussing technologies:
 - Explain Spring Boot, React, Angular, databases, APIs, authentication, deployment, and related concepts from first principles.
 - Compare options and explain trade-offs.
 - Recommend choices and provide justification that I can use during project reviews or interviews.
 - Once requirements are clear:
 - Design the system architecture.
 - Create frontend and backend module breakdowns.
 - Design the database schema.
 - Define API contracts.
 - Create an implementation roadmap.
 - When writing code:
 - Explain every file and folder.
 - Explain why the code is written that way.
 - Follow production-quality standards.
 - Mention alternative approaches and their trade-offs.
 - Assume I am learning while building. Your role is both solution architect and teacher.
 3. Important:
 - Never move to the next phase without explicitly asking for my approval.
 - If information is missing or ambiguous in the docs, identify it and ask clarifying questions.
 - Prioritize understanding and reasoning before coding.

- Help me prepare explanations that I can use to defend my design decisions to technical reviewers.
- 4. I approve of these steps. Below are the answers to your questions. We can move on to the next steps once to analyse my responses.

Will MongoDB be available locally, or should we use MongoDB Atlas/cloud?

Will CSV and JSON files be uploaded by users, or stored inside the backend project?

So at the moment this is a POC and we do not have real data only the data available in the picture. We can make our own data and work with that. This is an assignment that I've been given and I want to be the best in it. I want to think of all the loop holes.

Will CSV and JSON data change frequently? I assume yes but we will be making and using our own data.

Should the application reload CSV/JSON automatically when files change? idk what that means.

What should happen if the same customer exists in CSV but not MongoDB? Display an error ping but give whatever information is available about the customer.

What exact fields should be displayed in the customer list screen? As noted in the expected output atm i will update on the if anything changes.

Should the app have both:
customer list page, and
customer detail page? YES.

What export formats are expected: CSV, Excel, PDF, or all? All.

Should search work only by customer ID, or also by name, email, city, membership? Whatever you think is best.

Should users be able to edit customer data, or only view it? ONLY VIEW.

Does "AI-powered summary" require actual AI API integration? we can use a local llm for now like ollama.

Can we initially generate a rule-based summary? explain what it is. If u think its helpful use it.

Should the summary be generated on demand or stored in database? On demand.

Should the summary include spending behavior, membership, location, and preference? Yes.

Is login required? not atm.

Should there be roles such as Admin and User? not atm.

Should customer email/mobile be masked for some users? Sure but why?

Is there any mandatory database apart from MongoDB? I dont think so.

Should consolidated data be stored in another database, or generated dynamically every time? What do you think is best.

Should this project be deployed locally only, or on cloud? I want to commit code to git and run it on git free website services or on vercel.

Is Docker expected? no but if we can do it sure.

Is API documentation using Swagger/OpenAPI expected? This is a free no bidget project if any free resccources are available we can use them.
- 5. I approve of these steps. Below are the answers to your questions. We can move on to the next steps once to analyse my responses.

Okay lets do Option 1.

Option 1: Read files only when the application starts.

We can use a Hybrid Approach but keep ollama as main.

Should customer email/mobile be masked? Yes mask it.

Should consolidated data be stored or generated dynamically? Okay lets do dynamic generation.

6. Now that the requirements and approach are clear, help me design the complete Customer360 application architecture. I want a beginner-friendly explanation but it should still follow industry standards. Please explain how the backend, frontend, MongoDB, CSV file, JSON file, APIs, and AI summary flow will work together. Also explain why we are choosing Spring Boot, React, MongoDB, CSV/JSON file loading at startup, dynamic consolidation, and on-demand summary generation.
7. Create the backend project structure for Customer360 using Spring Boot. I am using IntelliJ and Java 25.0.1 with Spring Boot 4.1.0. Give me step-by-step instructions for creating the project, choosing dependencies, setting up Maven, and arranging packages. Explain every folder and file because I am new to this stack.
8. Help me create sample data for this POC. Since I do not have real customer data, create my own sample MongoDB customer profiles, CSV order data, and JSON customer preference data. Make sure all three sources can be joined using `customerId`. Include some cases where data is missing in CSV or JSON so that the application can show warnings and prove that the integration handles loopholes.
9. Build the Spring Boot backend models, DTOs, repositories, loaders, services, and controllers for the Customer360 application. The backend should read customer profile data from MongoDB, order data from CSV, and preference data from JSON. The service should dynamically consolidate all three sources using `customerId` instead of storing the final consolidated result in another database.
10. Add error/warning handling for missing data. If a customer exists in MongoDB but not in CSV or JSON, show whatever information is available and include a warning. If a customer exists in CSV but not in MongoDB, show an error ping or warning but still give whatever information is available where possible. The goal is to demonstrate that the application can handle real-world incomplete data.
11. Add customer list and customer detail APIs. The customer list should support search, filter, sort, and pagination. The customer detail API should show the complete consolidated profile including MongoDB profile data, CSV order data, JSON preference data, total order amount, membership, preferred channel, warnings, and data quality information.
12. Add export functionality to the backend. I want to export customer data in CSV, Excel, and PDF formats. Also support customer profile export for a selected customer. Explain which libraries are used, why they are selected, and how these exports work.
13. Add AI-powered customer summary functionality. For now, use a hybrid approach where Ollama/local LLM is the preferred approach, but also support a fallback rule-based summary if AI is not available. The summary should be generated on demand and should include spending behavior, membership, location, customer preference, and useful business insights.
14. Add Swagger/OpenAPI documentation for the backend because this is a free project and I want to use any free resources available for API testing and documentation. I should be able to open Swagger in the browser and test the APIs.
15. I am facing MongoDB local connection issues. The backend is trying to connect to `localhost:27017` and I am getting `MongoSocketOpenException` / cannot

connect to localhost. Help me understand the issue and guide me to fix it step by step.

16. I want to use MongoDB Atlas instead of local MongoDB. Help me configure MongoDB Atlas, create the database, set up the connection string, update Spring Boot configuration, and test that the backend is able to connect properly.
17. I am getting a MongoDB Atlas SSL or timeout error like
`javax.net.ssl.SSLException: Received fatal alert: internal_error` or `Timed out while waiting for a server that matches ReadPreferenceServerSelector`. Help me troubleshoot this issue.
18. The issue was with the IP address. MongoDB Atlas is now working after updating the allowed IP address. Continue from here and help me move to the next development step.
19. Build the React frontend for the Customer360 application. I want a modern dashboard with a customer list page and customer profile/detail page. The frontend should call the Spring Boot APIs, show consolidated data, and support search, filter, sort, pagination, export, AI summary, and warnings for missing data.
20. Design the Customer List dashboard UI. It should show total customers, page orders, page order amount, average order amount, membership customers, and customers without orders. It should have a header search box, filter panel, backend pagination table, status pills, membership pills, preferred channel pills, action buttons, and a customer profile section.
21. Add a Customer Profile section in the frontend. When I click a customer row or view icon, it should open the full customer profile with customer information, order summary, preferences, business metrics, recent orders, warnings, AI summary, and export actions.
22. Add frontend export actions. I want buttons for Export CSV, Export JSON, Export Excel, Export PDF, Export Profile CSV, and Print/PDF for the selected customer profile. Make sure the buttons are only shown to the correct users later when roles are added.
23. Add frontend styling in `App.css`. I want the design to look clean, professional, modern, and suitable for an assignment review. Use cards, grids, tables, pills, buttons, responsive layouts, and clear spacing. Keep the CSS understandable because I am learning.
24. I want to commit the current working code to GitHub. Guide me with the exact Git commands from the root folder, including `git status`, `git add`, `git commit`, and `git push`.
25. Now add authentication and login functionality to the application. I want a login page, backend login API, JWT authentication, protected frontend routes, logout, and switch user functionality. Use demo users for now.
26. Add role-based access control. Use three demo users: admin, manager, and viewer. ADMIN should have full access. MANAGER should be able to view dashboard, use AI summary, and export. VIEWER should only view dashboard and profile. Admin-only features should be hidden from Manager and Viewer.
27. Add backend authentication APIs: `/api/auth/login` and `/api/auth/me`. Add JWT token generation and validation. Add Spring Security configuration. Make sure protected APIs require a valid Bearer token.

28. Add frontend authentication files: `authApi.js`, `AuthContext.jsx`, `ProtectedRoute.jsx`, and `LoginPage.jsx`. Store the JWT token and user details in `localStorage`. Redirect unauthenticated users to `/login`. After successful login, redirect to `/customers`.
29. Update the customer API calls so that the frontend sends the JWT token in the Authorization header. Make sure customer list, customer details, AI summary, exports, admin data APIs, and customer management APIs use the token correctly.
30. Fix CORS because my frontend is running on Vite port `5174` instead of only `5173`. Update the backend security configuration to allow both `http://localhost:5173` and `http://localhost:5174`.
31. I got `403 Forbidden` while testing authenticated APIs from PowerShell. Help me understand why it happened and how to test correctly. I opened a new PowerShell window and `$login` was empty, so guide me to run login and API calls in the same window.
32. Give me PowerShell commands to test login, store the JWT token, create authorization headers, and call protected customer APIs.
33. Done. The login, JWT authentication, protected routes, logout, and role-based access control are working. Help me commit this phase.
34. I do not want to add the remaining filters now. Can we leave them as they are and move to the next important feature?
35. What would making a Dynamic Database mean? Cause the original requirement was to take 3 different data types for csv, mongodb and json. Explain what "dynamic database" should mean in this project without breaking the original requirement.
36. Let's make it dynamic. Keep the original three-source architecture: MongoDB for customer profile, CSV for orders, and JSON for preferences. But make CSV and JSON dynamic by allowing an admin to upload or refresh them from the UI/API without changing backend code.
37. Add admin backend APIs for dynamic CSV and JSON upload. ADMIN should be able to upload orders CSV and preferences JSON. The backend should refresh the in-memory cache immediately after upload. Add an admin status API to show current orders customer count and preferences customer count.
38. I uploaded `OrderCsvLoader.java`, `PreferenceJsonLoader.java`, `CustomerConsolidationService.java`, and `SecurityConfig.java`. Please update these files to support dynamic CSV/JSON upload and admin-only APIs while preserving the existing customer dashboard functionality.
39. Test the new admin data APIs using Swagger and PowerShell. Use `/api/auth/login` to get an admin JWT token, then call `/api/admin/data/status`, `/api/admin/data/upload/orders`, and `/api/admin/data/upload/preferences`.
40. I checked `/api/admin/data/status` and got `preferencesCustomerCount = 5`, `ordersCustomerCount = 6`, and message `Dynamic CSV and JSON cache is active`. Continue testing from here.
41. I uploaded test orders CSV with `C1001` and `C1002`, and the orders customer count changed to 2. Confirm that the dynamic upload is working and explain what should happen on the dashboard.

42. Create the frontend Admin Data Upload page. ADMIN should be able to upload Orders CSV and Preferences JSON from the UI, view current data status counts, go back to dashboard, and logout. MANAGER and VIEWER should not access this page.
43. Update React routing to include `/admin/data-upload`. Add an Upload Data button on the dashboard header for ADMIN only. Hide this button for MANAGER and VIEWER.
44. Yes it is opening. Can we commit it till here? Give me the Git commands to commit the dynamic CSV/JSON upload feature.
45. What are we doing next? Recommend the next most important development step after dynamic upload.
46. I want to update all the README files with the current project status and features.
47. Let's make the dynamic CSV/JSON upload persistent. Right now the upload works while the backend is running, but after restart it returns to default data. I want uploaded CSV/JSON files to be saved in the backend `uploads` folder and loaded again after restart.
48. Update the backend loaders so that startup behavior is: if uploaded files exist in `customer360-backend/uploads`, load those files; otherwise load default files from `src/main/resources/data`. Save uploaded orders as `customer_orders.csv` and uploaded preferences as `customer_preferences.json`.
49. Update `.gitignore` so uploaded runtime CSV and JSON files are not committed to GitHub. Add rules to exclude `customer360-backend/uploads/*.csv` and `customer360-backend/uploads/*.json`.
50. I do not understand where I am supposed to upload the data and what data I am supposed to upload. Explain clearly whether I upload files from the React UI or manually place files in the backend folder. Also give me sample test files for orders CSV and preferences JSON.
51. I tested the upload and the files are created in the backend uploads folder. `Test-Path` is returning true for both uploaded CSV and uploaded JSON files. Continue with the next validation step.
52. Done. Persistent upload is working after backend restart. Help me commit this feature.
53. Add Reset to Default Data. I want an ADMIN-only reset feature that deletes uploaded CSV/JSON files from the backend uploads folder, reloads the default CSV and JSON files from resources, refreshes the backend cache immediately, and updates the frontend status cards.
54. Update backend loaders with reset methods, update `AdminDataController` with `/api/admin/data/reset-defaults`, and update frontend `adminDataApi.js` and `AdminDataUploadPage.jsx` with a reset button and confirmation.
55. Give me the full updated `AdminDataUploadPage.jsx`. I already updated the CSS.
56. I tested reset using PowerShell and both uploaded files now return False with `Test-Path`. That means reset deleted the uploaded persistent files correctly. Confirm the expected default counts and help me commit this reset feature.

57. Done. The reset feature was committed with message `Add reset to default data feature`.
58. Update the README files again to include persistent uploaded CSV/JSON files, reset to default data, expected default counts, and the fact that uploaded files are ignored by Git.
59. Okay done. Let's move to the next feature.
60. Add Customer Profile CRUD in MongoDB. Earlier the app was view-only, but now I want ADMIN to be able to add, edit, and delete MongoDB customer profile data. MANAGER and VIEWER should not be able to add, edit, or delete.
61. Add backend Customer Profile CRUD APIs. Create a request DTO for customer profile input. Create an Admin Customer Controller with ADMIN-only endpoints for create, update, and delete. Required fields should include customerId for create, name, email, mobile, and city. Optional fields should include gender, age, dateOfBirth, address, and customerType.
62. Create `CustomerProfileRequest.java` under `customer360-backend/src/main/java/com/customer360/backend/dto`.
63. Create `AdminCustomerController.java` under `customer360-backend/src/main/java/com/customer360/backend/controller`.
64. Use the existing security rule where `/api/admin/**` is ADMIN-only. Do not unnecessarily change `SecurityConfig` if it already protects admin APIs.
65. Test the new customer profile create API using PowerShell. Create customer `C2001` with name, email, mobile, city, gender, age, dateOfBirth, address, and customerType.
66. I got `Unable to connect to the remote server`. Help me identify that the backend is not running and guide me to start the backend before testing the API.
67. I started the backend and tested create customer. The response says `C2001 Customer profile created successfully`. Continue with testing GET, PUT, and DELETE.
68. Give me PowerShell commands to test fetching `C2001`, updating `C2001`, and deleting `C2001`.
69. Done committed. Backend Customer Profile CRUD APIs are working and committed.
70. Now add the frontend Customer Profile CRUD UI. I want ADMIN-only actions for Add Customer, Edit Customer, and Delete Customer. MANAGER and VIEWER should not see or access these actions.
71. Provide me the full updated codes for the frontend customer management feature.
72. Create `customer360-frontend/src/api/adminCustomerApi.js` for create, update, and delete customer profile APIs.
73. Create `customer360-frontend/src/pages/AdminCustomerManagementPage.jsx`. It should include Add Customer form, Edit Customer mode, Delete confirmation, Refresh customer list, Back to Dashboard, Upload Data, Logout, and an important note that this page manages MongoDB profile data only while orders come from CSV and preferences come from JSON.
74. Update `App.jsx` to add the route `/admin/customers` protected by login and role checks.

75. Add a Manage Customers button in the dashboard header for ADMIN only. Place it near Upload Data and Logout.
76. I uploaded my latest `CustomerListPage.jsx`. Give me the full updated file with the ADMIN-only Manage Customers button added without breaking my existing dashboard, filters, exports, AI summary, profile view, pagination, RBAC, upload data, and logout logic.
77. Can you give me the fully updated `App.css`? I want the CSS to support the new Admin Customer Management page.
78. I uploaded my current `App.css`. Please merge the new Admin Customer Management CSS with my existing dashboard, login, upload, reset, and responsive styles. Do not break the existing design.
79. I want to build on this code. Can you update this and give me the full code. Use my current CSS as the base, and only add or merge what is needed for the Admin Customer Management page.
80. Okay done. How to test it now? Give me a clear end-to-end test plan for the new Admin Customer Management UI.
81. Test the full flow: start backend, start frontend, login as admin, open `/admin/customers`, add customer `C3001`, verify it appears in the dashboard, edit customer `C3001`, verify updates in dashboard, delete customer `C3001`, verify it is removed, then test MANAGER and VIEWER RBAC.
82. Doneeeee. The Admin Customer Management UI is working. Summarize what is completed and give me the Git commands to commit this phase.
83. I'd like to update the README and push it. I want full detailed README files.
84. Update the root `README.md` with the complete project overview, objective, architecture, data source design, features, routes, roles, demo users, run commands, testing flow, persistent upload behavior, reset behavior, admin customer management, Git ignore notes, current project status, and next recommended enhancements.
85. Update the backend `README.md` with the backend technology stack, package structure, authentication APIs, customer APIs, export APIs, admin data APIs, admin customer profile APIs, CSV/JSON upload formats, persistent upload storage, reset to default data, PowerShell test commands, MongoDB Atlas notes, Git ignore notes, and current backend status.
86. Update the frontend `README.md` with the frontend technology stack, folder structure, routes, login flow, logout flow, role-based UI, dashboard features, admin upload page, reset to default data, admin customer management page, testing instructions, CORS note, and current frontend status.
87. Provide the Git commands to commit and push the updated README files with a message like `Update detailed README documentation`.
88. I need a complete chronological record of all the prompts I used throughout this conversation to build this application. Please go through the entire chat from the beginning and extract every prompt or instruction that I gave which contributed to the application's design, implementation, architecture, UI/UX, backend, database, testing, deployment, documentation, or any other development-related work.

Requirements:

- Present the prompts as a single consolidated numbered list in chronological order.
- Preserve the original wording as closely as possible.
- Keep detailed prompts exactly as they were. Do not summarize, rewrite, or shorten them.
- For very small follow-up prompts (such as simple refinements, bug fixes, formatting requests, or one-line clarifications), condense them into concise single-line entries while preserving their intent.
- Merge consecutive prompts only when they relate to the same development task and can be understood as one request. Do not merge unrelated topics.
- Exclude non-development conversations (general chat, unrelated questions, casual discussion, etc.).
- Do not include explanations, analysis, commentary, or categorization—only the list.
- Ensure no development-related prompt is omitted.

The final output should be a clean, easy-to-review document that my manager can read from top to bottom to understand the progression of the application's development through my prompts.

87. Write each prompt in extreme detail. Like how I wrote previously. Expand the chronological prompt record into detailed prompt-style entries that clearly show the full progression of the application from requirement understanding to architecture, backend, frontend, authentication, RBAC, dynamic data upload, persistence, reset, CRUD, UI, testing, Git commits, and documentation.